

Project Design Phase:

Solution Architecture

1. Solution Architecture Diagram

The diagram below details the multi-layered architecture of FloodSense AI, mapping user interactions from the client-side presentation layer through the Flask backend controllers, the machine learning inference engine, and the persistent relational database.

2. Component Breakdown

The architecture is divided into four decoupled tiers, each serving a specific structural and behavioral role:

A. Presentation Layer (Web Client)

- **Structure:** Built using lightweight, semantic HTML5 structure, styled with premium dark-mode glassmorphic CSS, and driven by asynchronous JavaScript.
- **Behavior:**
 - Intercepts form submission, packages the 10 meteorological and precipitation inputs into a JSON payload, and makes an asynchronous `fetch()` call to the `/predict` API.
 - Dynamically updates the DOM on API response to animate the risk gauge bar and inject list directives corresponding to safety alerts.
 - Decodes SQLite timestamp records and formats dates dynamically based on the local client timezone.

B. Application Layer (Flask Web Server)

- **Structure:** A Python Flask microservice controller (`app.py`) managing routes, security context, and API endpoints.
- **Behavior:**
 - **Routing:** Serves client views and handles database transaction requests.
 - **Session Management:** Keeps track of login status and roles (Admin or User) using signed cookies.
 - **Flask application context (g):** Instantiates database connections dynamically at the start of a request and guarantees automatic cleanup via `teardown_appcontext` hooks to prevent connection locks.

C. Intelligence Layer (Inference Engine)

- **Structure:** Joblib deserializers loading pre-trained ML parameters (`scaler.joblib` and `flood_model.joblib`) on application startup.
- **Behavior:**

- Transforms raw inputs into unified scales via StandardScaler to handle features with wide boundaries (e.g., Annual Rainfall vs. Temperature).
- Executes predictions and extracts class probabilities via the Random Forest algorithm to compute exact percentages for high-risk versus safe nominal levels.

D. Persistence Layer (Database)

- Structure: Relational SQLite database (floodsense.db) housing four associated schemas (users, ml_models, weather_data, and prediction_results).
- Behavior:
 - Provides secure, index-optimized lookups for login requests.
 - Stores input environmental variables mapping directly to the user profile.
 - Tracks predictions and classifications in a structured history log for audit trails.

3. Data and Behavioral Flow

When a logged-in user requests a prediction, the data flows as follows:

[User Input] → [JS Fetch] → [Flask Route] → [Standard Scaler]

|



[Web Dashboard] ← [JSON Res] ← [SQL Logging] ← [Random Forest Model]

1. Input Submission: The user enters the 10 weather indicators and clicks "Analyze Flood Risk".
2. API Transport: Client-side JS validates the inputs and posts them to the /predict controller endpoint.
3. Data Standardizing: The backend receives the JSON data, formats it into a Pandas DataFrame, and applies standard scaling based on the training dataset's metrics.
4. Inference Execution: The normalized vectors are passed to the Random Forest model to calculate prediction targets (Flood / No Flood) and probability confidence.
5. Logging & Persistence: The user input metrics are recorded in weather_data, and the resulting classifications are linked and saved into prediction_results.
6. Response & Presentation: The endpoint returns a JSON payload. The frontend updates the dashboard gauge and pulls in real-time safety recommendations based on the risk category.